VOLE-ACT: Post-Quantum Anonymous Credit Tokens from VOLE-in-the-Head and MAYO

Samuel Schlesinger

July 2026

Research draft — definitions and analysis are stated as targets;
no complete security reduction is claimed

Abstract

Anonymous credit tokens let an issuer grant a client a hidden, spendable balance: the client can later redeem credits unlinkably, each redemption consumes a one-time nullifier and returns a fresh token on the remaining balance. Classical constructions rest on discrete-logarithm-style algebra — blind or randomizable signatures, algebraic MACs, Sigma protocols — none of which survives a quantum adversary. We present VOLE-ACT, a post-quantum instantiation of the Katz–Schlesinger anonymous-credit design [20] whose entire symmetric layer is Keccak and whose only algebraic assumption concerns the multivariate-quadratic (whipped MAYO) trapdoor. The central idea is to *avoid blindness entirely*: the issuer signs a revealed SHAKE256 commitment digest whose preimage stays hidden, and possession of the signature is later proven in zero knowledge inside a VOLE-in-the-Head proof system that is native to exactly the two ingredients that must be proven — Keccak (a Boolean circuit) and MAYO verification (a quadratic system over $\mathbb{F}_{16} \subseteq \mathbb{F}_{2^{128}}$). We give syntax and security definitions for anonymous credit tokens, the construction — including a degree-16 Keccak checkpointing technique that commits the permutation state only every four rounds, and a batched quadratic-system constraint for whipped MQ verification — and a security analysis that states its assumptions (an adaptive one-more-preimage property of the whipped map; an ideal-XOF model) and its concrete attack ceilings explicitly. A Rust research prototype achieves 19–20 ms proving and 6.5–8 ms issuer verify-and-sign (proof verification plus MAYO preimage sampling) per spend, with 53 KiB proofs and 283-byte tokens.

1 Introduction

An anonymous credit token system has one issuer and many clients. The issuer grants a client a public number of credits; the client can later spend part of that balance without revealing which issuance — or which earlier spend — produced the token being presented. Each accepted spend consumes a one-time nullifier (preventing double spending) and returns a fresh token carrying the remaining balance. The functionality generalizes anonymous tokens in the Privacy Pass style [18] from unit tokens to stateful balances, and specializes keyed-verification anonymous credentials [17] to a single numeric attribute with exact arithmetic.

Classically this is well-trodden ground: blind signatures [15], CL signatures [16], algebraic MACs [17] and their pairing-based relatives give compact, efficient constructions. All of them lean on the algebraic structure of discrete-log or pairing groups, both for the signature and for the efficient zero-knowledge presentation protocol. Neither survives a quantum adversary. The

natural repair — a post-quantum *blind* signature — remains an active and unsettled research area in which several early lattice-based proposals required substantial revision after cryptanalysis; we prefer a design that does not need blindness at all.

The core idea: sign a digest, hide the preimage. VOLE-ACT, following the Katz–Schlesinger design [20], sidesteps blindness. A credential is a short XOF stream

$$X \;=\; \mathsf{SHAKE256}(\text{domain} \,\|\, \mathsf{ctx} \,\|\, \kappa \,\|\, \mathsf{enc}_{64}(b) \,\|\, \rho),$$

over a client-chosen 256-bit nullifier key κ, public balance b, and 256-bit hiding nonce ρ. The first $4m$ bits of X form the *commitment digest* $\mathsf{C} \in \mathbb{F}_{16}^m$; the next 256 bits form the *nullifier* N. At issuance the client *reveals* C and proves in zero knowledge that it is well-formed for the public balance b; the issuer then computes an ordinary MAYO preimage $\sigma \leftarrow \mathsf{SPre}(\mathsf{sk}, \mathsf{C})$ and returns it. Nothing is blind: the issuer sees exactly the digest it signs. Unlinkability comes later — at spend time the client proves *in zero knowledge* that it knows $(\sigma, \kappa, b, \rho)$ with $P^*(\sigma) = \mathsf{C}(\kappa, b, \rho)$, revealing only the nullifier N, the public spend amount, and a fresh commitment digest C' for the successor token. The signature and the signed digest are never shown again, so there is nothing to link. The unrevealed suffix of X is treated as pseudorandom given its prefix, which is what makes the nullifier unlinkable to the issuance in which the issuer saw C.

This pattern reduces the required primitives to (i) a post-quantum trapdoor map whose *verification* is cheap inside a proof system, and (ii) a post-quantum zero-knowledge proof system that is efficient on *hash-heavy Boolean* statements. The pairing of MAYO with VOLE-in-the-Head is unusually harmonious on both counts:

- **MAYO** [1, 3] verification evaluates a public multivariate quadratic map over $\mathbb{F}_{16}$. Since $\mathbb{F}_{16}$ embeds as a subfield of the proof system's tag field $\mathbb{F}_{2^{128}}$, the whole verification equation becomes a single batched *quadratic system* constraint in the proof — no bit decomposition, no modular arithmetic emulation.
- **VOLE-in-the-Head** [4, 5] proof systems commit witness *bits* and prove Boolean and low-degree relations at QuickSilver [7, 8] cost: linear gates are free, and the entire batch of polynomial constraints of maximum degree d is checked with d published $\mathbb{F}_{2^{128}}$ elements. Keccak — the only hash used anywhere — is the friendliest possible circuit for such a system: its round function is degree 2.

Contributions. (1) Syntax and game-based security definitions for anonymous credit tokens with issuer-settled (*deferred-return*) spends (Section 3), including a candidate semi-formal statement of the assumption the issuer's preimage-oracle role forces: an adaptive *one-more-preimage* property of the whipped MAYO map. (2) A post-quantum construction from that assumption plus an ideal XOF (Section 4): one XOF stream yielding both commitment and nullifier; a typed two-format token state machine; and a deferred-return settlement in which the issuer selects a bounded return only *after* verifying the spend proof, bound by a nested hash. (3) Proof-circuit techniques (Section 5): *degree-16 Keccak checkpointing*, committing the permutation state only every four rounds and quartering the dominant witness cost by spending the proof system's polynomial-degree budget; and a batched quadratic-system constraint expressing whipped MQ verification over shared product terms, folded by verifier randomness. (4) A security analysis (Section 6) that is explicit about its conditional status, its concrete error terms, and its generic attack ceilings — including a $2^{256/3}$-query quantum collision bound on

the credential digest. (5) Evidence of practicality: a Rust research prototype (Section 7) with 19–20 ms spend proving, 6.5–8 ms issuer verify-and-sign, 53 KiB compact proofs, and 283-byte tokens.

2 Preliminaries

2.1 Notation

$\lambda = 128$ throughout. $\mathbb{F}_2$, $\mathbb{F}_{16} = \mathbb{F}_2[x]/(x^4+x+1)$ and $\mathbb{F}_{2^{128}} = \mathbb{F}_2[x]/(x^{128}+x^7+x^2+x+1)$ are the Boolean, MAYO, and tag fields; $\iota : \mathbb{F}_{16} \hookrightarrow \mathbb{F}_{2^{128}}$ is the protocol-specified field embedding (the subfield is unique; the embedding is fixed deterministically, with the reduction polynomial family used by FAEST [5]). SHAKE256 is used for every PRG, commitment, and Fiat–Shamir hash, always with prefix-free domain separation and fixed-width or length-framed inputs [13]. $\mathsf{enc}_{64}(\cdot)$ is 64-bit little-endian encoding. κ denotes the client's nullifier key, h a GGM tree depth, and k_w MAYO's whipping parameter.

2.2 VOLE-based zero knowledge and VOLE-in-the-Head

In VOLE-based ZK [8, 7], the prover holds, for each witness bit $u_t \in \{0,1\}$, a *tag* $V_t \in \mathbb{F}_{2^{128}}$, and the verifier holds a global secret $\Delta \in \mathbb{F}_{2^{128}}$ and per-coordinate *keys*

$$K_t \;=\; V_t + u_t \cdot \Delta,$$

a vector oblivious linear evaluation (VOLE) correlation. Wires are affine combinations of committed bits and are linearly homomorphic for free. A multiplication (degree-2) constraint $a \cdot b = c$ is checked QuickSilver-style: the verifier forms $B = K_a K_b + \Delta K_c$, which the prover can match with a degree-1 polynomial in Δ exactly when the constraint holds; a random linear combination with challenge weights χ_i batches all constraints into one polynomial identity checked at Δ. The generalization to degree-d polynomial constraints costs d published coefficients and $d-1$ random VOLE masks in total for the whole batch.

VOLE-in-the-Head [4] makes this *publicly verifiable and non-interactive*: the prover generates the VOLE correlation itself from τ GGM [11] seed trees of depth h (with $\tau \cdot h = \lambda$), commits to all 2^h leaves of each tree with an all-but-one vector commitment, and the Fiat–Shamir challenge Δ selects, per tree, one leaf that stays hidden; the remaining leaves are opened and the verifier recomputes its keys. Soundness comes from the hidden leaf; zero knowledge from the fact that the opened leaves are exactly what the verifier can recompute anyway. Our instantiation follows the hardening of FAEST [5] and "One Tree to Rule Them All" [6]: every PRG and commitment call binds a per-proof 2λ-bit salt, a per-tree derived salt, and the exact tree position, preventing the known multi-target and multi-instance amortization attacks on the 128-bit seeds; small-field consistency across the τ trees uses public corrections re-based onto one shared u-vector, in the style of SoftSpokenOT [10].

Three parameter profiles trade proof size against tree-expansion work: *compact* $(\tau, h) = (16, 8)$, *balanced* $(32, 4)$, and *low-latency* $(64, 2)$. Each opened proof reveals τ tree openings of h sibling seeds, so proofs shrink as h grows while the prover expands $\tau \cdot 2^h$ leaves.

The consistency check must be a matrix hash. Because each tree contributes only h bits of Δ, a cheating prover who mangles one tree's correction data is caught by a per-tree check

only with probability $1 - 2^{-h}$ — as low as 3/4 for $h = 2$. The construction therefore applies a 128-row *column-wise linear universal hash* to the entire tag matrix (two independent polynomial evaluations combined with random weights, plus a one-time-pad block), so that any inconsistency survives into the check equation except with probability $\approx 2^{-\lambda}$ plus small degree-dependent evaluation terms, regardless of the chunk structure (an exact bound is part of the outstanding analysis). A scalar hash in place of the matrix hash is a real soundness bug we caught during development; we flag it because the failure is silent and parameter-dependent.

2.3 MAYO

MAYO [1, 3] is an oil-and-vinegar [12] trapdoor map: a public quadratic map $P : \mathbb{F}_{16}^{n} \to \mathbb{F}_{16}^{m}$ vanishing on a secret o-dimensional oil space, with $o < m$ deliberately too small for direct Kipnis–Shamir-style attacks; the "whipping" construction evaluates k_w correlated copies to stretch the oil space, so the effective (whipped) public map is $P^* : \mathbb{F}_{16}^{k_w n} \to \mathbb{F}_{16}^{m}$, matching the notation of the MAYO literature. The trapdoor samples preimages σ with $P^*(\sigma) = y$ for arbitrary targets y; this *hash-then-invert* usage (rather than MAYO's message-signing API) is exactly what the scheme needs. We use the round-2 parameter set MAYO_2: $(n, m, o, k_w) = (81, 64, 17, 4)$, so $\sigma \in \mathbb{F}_{16}^{324}$ (162 bytes) and targets are $m = 64$ nibbles (32 bytes). We note the risk posture explicitly: multivariate trapdoors have a difficult history — Rainbow fell to a practical attack in 2022 [2] — and MAYO's whipping is comparatively young. The design compartmentalizes this risk: a MAYO break forges credentials but is not expected to deanonymize past transactions, whose privacy rests only on the hash-based layer (Claim 2).

For the proof circuit, the whipped map is expanded once per public key into m upper-triangular forms $G_a \in \mathbb{F}_{16}^{k_w n \times k_w n}$ with $P^*(\sigma)_a = \sigma^\top G_a \, \sigma$; the circuit then proves m quadratic equations over the *same* $\binom{k_w n + 1}{2}$ product terms (Section 5.2).

3 Definitions and assumptions

We state the syntax and the security games the eventual analysis must target; a fully formal treatment of concurrent sessions and adaptive corruptions is left to the reduction effort. Throughout the experiments: the application label app and the parameter, profile, and balance-width identifiers are fixed public constants; KGen is MAYO trapdoor generation; $\mathsf{CanonParse}$ is the scheme's canonical public-key decoder, returning $\perp$ on any non-canonical encoding; and $\mathsf{DeriveCtx}(\mathsf{pk})$ is Setup's context derivation applied to those constants and pk. The maps C, D, digest, and $\mathsf{DeriveCtx}$ are all defined over one shared prefix-consistent ideal XOF $\mathcal{H}$ (Remark 4), which every experiment implicitly samples and provides to all parties, including the adversary.

Definition 1 (Anonymous credit token scheme)**.** *An* anonymous credit token scheme *with balance space* $\{0, \ldots, 2^{64}-1\}$ *is a tuple of interactive algorithms*

$$(\mathsf{Setup},\ \mathsf{Issue},\ \mathsf{Spend},\ \mathsf{SpendDeferred},\ \mathsf{VerifyToken})$$

between an issuer and clients:

- $\mathsf{Setup}(1^\lambda, \mathsf{app}) \to (\mathsf{sk}, \mathsf{pk}, \mathsf{ctx})$*: generates the issuer trapdoor, public key, and a context binding the key, application domain, parameter set, and protocol version.*
- $\mathsf{Issue}\langle \mathrm{C}(\mathsf{pk}, b), \mathrm{I}(\mathsf{sk}, b)\rangle \to T$ *(public balance* b*): the client obtains a* direct token T *of effective balance* b*, or aborts.*

- $\mathsf{Spend}\langle \mathrm{C}(\mathsf{pk}, T, s), \mathrm{I}(\mathsf{sk})\rangle \to T'$ *(public amount s): consumes T's nullifier and yields a direct token T' of effective balance* $\mathsf{bal}(T) - s$*, or aborts.*
- $\mathsf{SpendDeferred}\langle \mathrm{C}(\mathsf{pk}, T, s), \mathrm{I}(\mathsf{sk}, t')\rangle \to T'$*: as* Spend*, except the issuer selects a return $t' \le s$* after *verifying the client's proof; the output is a* deferred-return token *of effective balance* $\mathsf{bal}(T) - s + t'$*.*
- $\mathsf{VerifyToken}(\mathsf{pk}, T) \to \{0, 1\}$*: local client authentication of a decoded token. Tokens are never sent as presentations; only spend transcripts are.*

Both spend operations accept either token format as input; the four input/output transitions carry distinct type tags bound into their transcripts.

Definition 2 (Correctness, informal)**.** *Consider an experiment in which an environment drives an honest issuer and honest clients to build a DAG of tokens: roots arise from* $\mathsf{Issue}(b)$ *and every internal node from spending a not-yet-spent node, with amounts respecting balances. The scheme is correct if, except with negligible probability, no honest operation aborts, every client-side authentication check passes, every output balance obeys the equations of Definition 1, distinct accepted inputs yield distinct nullifiers, and exact retransmission of a request returns the stored response rather than a new node.*

Definition 3 (Fiscal soundness)**.** *For a scheme* Π *and adversary* $\mathcal{A}$*, the experiment* $\mathrm{Exp}^{\mathrm{fiscal}}_{\Pi,\mathcal{A}}(\lambda)$ *of Figure 1 runs* $\mathcal{A}$ *as all clients against an honest issuer.* Π *is* fiscally sound *if for every efficient* $\mathcal{A}$*,* $\mathrm{Adv}^{\mathrm{fiscal}}_{\Pi,\mathcal{A}}(\lambda) := \Pr[\mathrm{Exp}^{\mathrm{fiscal}}_{\Pi,\mathcal{A}}(\lambda) = 1]$ *is negligible.*

Remark 1 (Redemption-only is without loss of generality)**.** *The win condition counts only* redeemed *value, not outstanding spendable balances, so an attack that merely* forks *a token into two live successors has not yet won. It extends to a win at will: any adversary holding more spendable value than the invariant allows simply continues querying* oSpend *to cash the excess out, and the game runs for the whole experiment. Formally, redemption-only soundness is equivalent to the ledger invariant of the design document (redeemed value + live independently spendable value ≤ issued value) up to this cash-out reduction, which is the statement a full analysis should prove as a lemma rather than assume.*

Definition 4 (Spend anonymity, single challenge)**.** *For a scheme* Π *and adversary* $\mathcal{A}$ *(a malicious issuer), the experiment* $\mathrm{Exp}^{\mathrm{anon}}_{\Pi,\mathcal{A}}(\lambda)$ *of Figure 2 is defined with the convention that the experiment's explicit abort steps (an invalid public key or an invalid challenge selection) output a uniformly random bit, so degenerate strategies gain nothing; a malformed final guess is likewise replaced by a fresh coin. Aborts the adversary induces* inside *protocol sessions are not experiment aborts.* Π *is* spend-anonymous *if for every efficient* $\mathcal{A}$*,* $\mathrm{Adv}^{\mathrm{anon}}_{\Pi,\mathcal{A}}(\lambda) := \left|\Pr[\mathrm{Exp}^{\mathrm{anon}}_{\Pi,\mathcal{A}}(\lambda) = 1] - \frac{1}{2}\right|$ *is negligible.*

Remark 2 (Design of the anonymity game)**.** *The same-kind, same-mode, same-amount, equal-balance restrictions reflect leakage the syntax makes public by design. Retiring the* candidates *closes the immediate distinguisher: replaying a candidate would reveal β through nullifier reuse. Retiring the* successor *(whose balance is the same in both worlds, given equal candidate balances) is not needed against that attack; it isolates the definition to a single challenged transcript rather than smuggling in a compositional guarantee about continued use. A chain definition (hidden, permuted token handles with identical admissible continuations) is the natural strengthening and is left open. Section 6 discusses the metadata any deployment leaks outside this game.*

$\mathrm{Exp}^{\mathrm{fiscal}}_{\Pi,\mathcal{A}}(\lambda)$

$(\mathsf{sk}, \mathsf{pk}, \mathsf{ctx}) \leftarrow\$ \mathsf{Setup}(1^\lambda, \mathsf{app})$
$I \leftarrow 0; \quad R \leftarrow 0; \quad \mathsf{St} \leftarrow \emptyset; \quad \mathsf{win} \leftarrow 0$
run $\mathcal{A}^{\mathsf{oIssue},\mathsf{oSpend},\mathsf{oSpendDef}}(\mathsf{pk}, \mathsf{ctx})$
return win

$\mathsf{oIssue}(b)$

run the issuer side of Issue at balance b with $\mathcal{A}$ as client
if the issuer accepts: $I \leftarrow I + b$; **return** its response
return $\perp$

$\mathsf{oSpend}(req)$

if *req* does not parse as a typed ordinary-settlement request:**return** $\perp$
$(\mathsf{N}, s) \leftarrow$ its nullifier and amount; $dig \leftarrow \mathsf{digest}(\mathsf{ctx}, req)$
if $\mathsf{St}[\mathsf{N}] = (dig', resp')$ exists:
  if $dig' = dig$: **return** $resp'$ // exact retry: no accounting change
  return $\perp$ // conflicting reuse of a consumed nullifier
if issuer verification of *req* fails:**return** $\perp$
$resp \leftarrow$ issuer response (fresh preimage on the fresh target)
$\mathsf{St}[\mathsf{N}] \leftarrow (dig, resp)$; $R \leftarrow R + s$
if $R > I$: $\mathsf{win} \leftarrow 1$
return *resp*

$\mathsf{oSpendDef}(req, t')$

if *req* does not parse as a typed deferred-settlement request:**return** $\perp$
$(\mathsf{N}, s) \leftarrow$ its nullifier and amount; $dig \leftarrow \mathsf{digest}(\mathsf{ctx}, req)$
if $\mathsf{St}[\mathsf{N}] = (dig', resp')$ exists:
  if $dig' = dig$: **return** $resp'$ // exact retry replays the stored t'; accounting unchanged
  return $\perp$
if issuer verification of *req* fails $\vee$ $\neg(0 \le t' \le s)$: **return** $\perp$
$resp \leftarrow$ issuer response on the deferred target, recording t'
$\mathsf{St}[\mathsf{N}] \leftarrow (dig, resp)$; $R \leftarrow R + s - t'$
if $R > I$: $\mathsf{win} \leftarrow 1$
return *resp*

Figure 1: The fiscal-soundness experiment. Accounting is per logical spend event (first durable consumption of a nullifier); letting $\mathcal{A}$ choose $t' \le s$ models issuer return policy conservatively. External redemption of value must be idempotent at the same store boundary.

Assumption 1 (Adaptive one-more preimage for whipped MAYO)**.** *For an adversary $\mathcal{A}$ making at most Q_H (classical) queries to the ideal XOF $\mathcal{H}$ and at most Q_S calls to* oInv*, the experiment* $\mathrm{Exp}^{\mathrm{omp}}_{\mathcal{A}}(\lambda)$ *of Figure 3 is defined over the two admissible target languages the protocol signs. The*

$\mathrm{Exp}^{\mathrm{anon}}_{\Pi,\mathcal{A}}(\lambda)$

$\beta \leftarrow\$ \{0,1\}$; $\mathcal{T} \leftarrow \emptyset$ // handle table of honest tokens
$\mathsf{pk}_{\mathrm{bytes}} \leftarrow\$ \mathcal{A}(1^\lambda)$
$\mathsf{pk} \leftarrow \mathsf{CanonParse}(\mathsf{pk}_{\mathrm{bytes}})$; **if** $\mathsf{pk} = \bot$: abort
$\mathsf{ctx} \leftarrow \mathsf{DeriveCtx}(\mathsf{pk})$ // as in Setup; fixed hereafter
$(h_0, h_1, s, \mathsf{mode}) \leftarrow\$ \mathcal{A}^{\mathsf{oClientOp}}()$
if $\neg\mathsf{ValidChallenge}(h_0, h_1, s, \mathsf{mode})$: abort
run the honest client side of the mode-mode spend of amount s on $\mathcal{T}[h_\beta]$,
  with $\mathcal{A}$ as issuer; $\mathcal{A}$ sees the transcript, and may abort the session
if the session accepts: $\mathsf{spent}[h_\beta] \leftarrow 1$; store the successor at a fresh handle h^*; $\mathsf{retire}(h^*)$
$\mathsf{retire}(h_0)$; $\mathsf{retire}(h_1)$ // an issuer-induced challenge abort stays in the game
$\beta' \leftarrow\$ \mathcal{A}^{\mathsf{oClientOp}}()$; **if** $\beta' \notin \{0,1\}$: $\beta' \leftarrow\$ \{0,1\}$
return $[\beta' = \beta]$

$\mathsf{oClientOp}(\mathsf{op}, h, \mathit{args})$

if $\mathsf{op} \notin \{\mathsf{Issue}, \mathsf{Spend}, \mathsf{SpendDeferred}\}$: **return** $\bot$
if $\mathsf{op} \neq \mathsf{Issue} \wedge (h \notin \mathcal{T} \vee \mathsf{retired}[h] \vee \mathsf{spent}[h])$: **return** $\bot$
run the honest client side of op on $\mathcal{T}[h]$ where required, with $\mathcal{A}$ as issuer,
  performing every client-side authentication check
if the session accepts:
  if $\mathsf{op} \neq \mathsf{Issue}$: $\mathsf{spent}[h] \leftarrow 1$
  store the output token at a fresh handle h'; **return** h'
return $\bot$ // aborted sessions yield no token and consume no handle

$\mathsf{ValidChallenge}(h_0, h_1, s, \mathsf{mode})$

return $\mathsf{mode} \in \{\mathsf{Spend}, \mathsf{SpendDeferred}\} \wedge h_0 \neq h_1$
  $\wedge$ $\mathcal{T}[h_0], \mathcal{T}[h_1]$ exist, $\neg\mathsf{retired}, \neg\mathsf{spent}$, authenticated
  $\wedge$ same credential kind $\wedge$ $\mathsf{bal}(\mathcal{T}[h_0]) = \mathsf{bal}(\mathcal{T}[h_1])$ $\wedge$ s valid for both

Figure 2: The single-challenge spend-anonymity experiment against a malicious issuer. The explicit "abort" steps (invalid key, invalid challenge selection) output a uniformly random bit; an issuer-induced abort of the challenge *session* is ordinary protocol behavior — the transcript stands and $\mathcal{A}$ still guesses, so selective-failure strategies remain in scope. All handle state (spent, retired) initializes to 0.

assumption is that for every efficient $\mathcal{A}$, $\mathrm{Adv}^{\mathrm{omp}}_{\mathcal{A}}(\lambda) := \Pr[\mathrm{Exp}^{\mathrm{omp}}_{\mathcal{A}}(\lambda) = 1]$ *is negligible (as a function of* Q_S *and* Q_H*).*

Remark 3 (Modeling choices in Assumption 1)**.** *The opening-accompanied oracle mirrors the protocol, where the issuer inverts only targets carrying a verified proof of opening knowledge, and* resamples *on repeated requests: only exact spend retransmissions are answered from a cache, which the game models as no query at all, and preimages the issuer samples but discards after losing an insertion race never leave the issuer and are not oracle replies. Counting* distinct *targets in the*

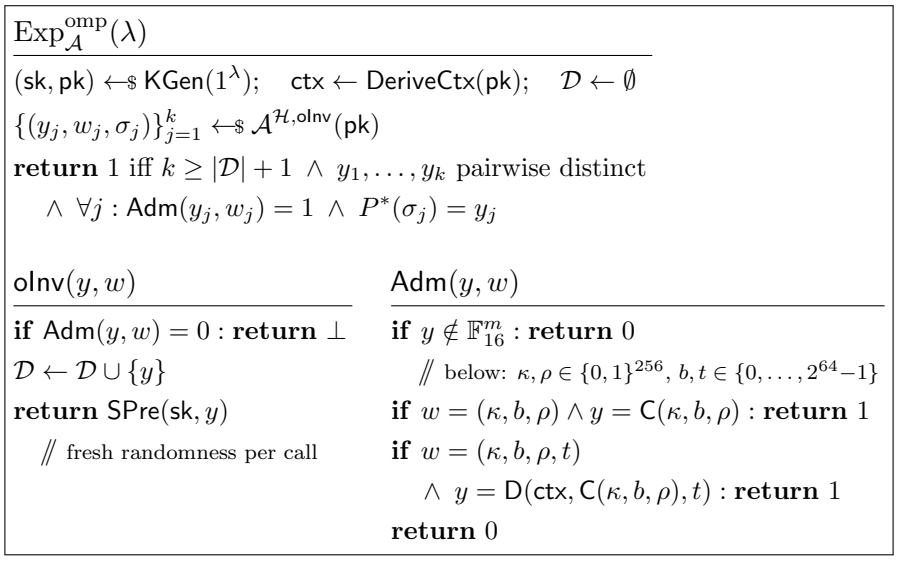

Figure 3: The adaptive one-more-preimage experiment. $\mathcal{H}$ is the ideal (prefix-consistent) XOF; C and D are the credential and deferred target maps under their domain tags. Success requires $|\mathcal{D}|+1$ pairwise-distinct admissible targets, where $\mathcal{D}$ is the set of *distinct* queried targets; the total call budget $Q_S \geq |\mathcal{D}|$ enters only concrete bounds.

success threshold is essential: charging every call would let r resamplings of one target excuse a single forged new target, which already violates fiscal soundness. The assumption is stronger and less studied than MAYO's signature unforgeability — the issuer here is *a preimage oracle. PoMFRIT [19] formalizes an (n, q) variant with n* random *targets, at most q inversion calls, and $q+1$ distinct target indices for success; bridging that game to the adaptive, opening-accompanied, two-language version above (and stating a QROM variant with quantum XOF access) is part of the outstanding analysis, not a citation we inherit.*

Remark 4 (Hash modeling and adversary class)**.** *The analysis models* SHAKE256 *as an ideal XOF — a* prefix-consistent *variable-output random oracle, so that reading $4m{+}256$ bits and reading $4m$ bits of the same stream agree on their common prefix (ordinary length-indexed random-oracle outputs do not automatically provide this) — for three distinct roles: the credential stream (hiding, collision and preimage structure, suffix pseudorandomness given a prefix), the GGM/vector-commitment PRG layer, and Fiat–Shamir. FIPS 202 standardizes SHAKE but proves none of these properties; sponge indifferentiability is the standard heuristic bridge. The claims of Section 6 target* classical *PPT adversaries in this ROM; the primitives are post-quantum candidates, but a QROM treatment (quantum XOF access, quantum rewinding for extraction) is explicitly open, so "post-quantum" describes the assumption class, not a proved quantum security level.*

4 The VOLE-ACT construction

4.1 Credentials and nullifiers from one XOF stream

Fix a 256-bit context ctx binding the issuer key (by hash of the expanded public map), application domain, parameter set, VOLE profile, and protocol version. The client samples $\kappa, \rho \leftarrow \{0,1\}^{256}$

and, for base balance $b \in \{0, \ldots, 2^{64}-1\}$, derives the stream

$$X = \mathsf{SHAKE256}(\text{“credential”} \,\|\, \mathsf{ctx} \,\|\, \kappa \,\|\, \mathsf{enc}_{64}(b) \,\|\, \rho), \quad \mathsf{C} = X[0:4m], \quad \mathsf{N} = X[4m:4m{+}256].$$

For MAYO$_2$, C is $4m = 256$ bits; its width as a collision domain matters for fiscal security (Section 6). Both values come from a *single* sponge evaluation — the message fits one Keccak rate block — so proving both costs one in-circuit permutation. The commitment prefix C is revealed to the issuer exactly once (at signing time); the nullifier suffix N is revealed exactly once (at spending time). Under the ideal-XOF modeling, the suffix is pseudorandom given the prefix, which is what divorces the spend from the issuance.

4.2 Two token formats and the settlement state machine

A *direct* token is $(\sigma, \kappa, b, \rho)$ with $P^*(\sigma) = \mathsf{C}(\kappa, b, \rho)$ and effective balance b. A *deferred-return* token additionally carries an issuer-chosen return t, with

$$P^*(\sigma) \;=\; \mathsf{D}\big(\mathsf{ctx}, \mathsf{C}(\kappa, b, \rho), t\big) \;=\; \mathsf{SHAKE256}\big(\text{“deferred”} \,\|\, \mathsf{ctx} \,\|\, \mathsf{pack16}(\mathsf{C}) \,\|\, \mathsf{enc}_{64}(t)\big)[0:4m],$$

and effective balance $b + t$ (exact, non-wrapping). (The formal tuples elide fixed metadata the implementation stores and VerifyToken checks: the derived context, the credential-kind tag, and the direct format's structurally zero top-up.) The nested hash gives a simple, domain-separated binding of both the base commitment and the return: changing t requires a fresh preimage or a target collision, and no algebraic combiner (such as $\mathsf{C} + \iota(t)$) has to be analyzed for malleability at all.

Spends are typed. An ordinary Spend consumes either format and always outputs a direct token; SpendDeferred consumes either format and outputs a deferred-return token. The client commits to a fresh (κ', ρ', b') *before* the issuer chooses $t' \le s$, which is precisely the case that cannot be expressed as a smaller net spend — if the return were known in advance, the client would simply spend less. All four transitions carry distinct domain-separation tags end-to-end, so mode confusion at the byte level cannot create an untyped path.

4.3 Algorithms

Figure 4 instantiates Definition 1. pack16 packs a $\mathbb{F}_{16}$-vector two nibbles per byte, first element in the low nibble; all domain tags are ASCII strings under prefix-free framing; Π.Prove/Verify is the VOLEitH argument of Section 5 for the stated relation, with the statement (all public values, tags, and ctx) bound into Fiat–Shamir.

4.4 The spend relation

To spend public amount s from a direct token, the client reveals $(s, \mathsf{N}, \mathsf{C}')$ and proves knowledge of $(\sigma, \kappa, b, \rho, \kappa', b', \rho')$ such that

$$P^*(\sigma) = \mathsf{C}(\kappa, b, \rho), \qquad \mathsf{N} = \mathsf{N}(\kappa, b, \rho), \qquad b' + s = b, \qquad \mathsf{C}' = \mathsf{C}(\kappa', b', \rho').$$

For a deferred-input spend the first equation becomes $P^*(\sigma) = \mathsf{D}(\mathsf{ctx}, \mathsf{C}(\kappa, b, \rho), t)$ with hidden t, together with the exact fold $b + t = e$ and $b' + s = e$. The balance equations are *exact 64-bit unsigned additions with zero final carry*, proven over committed carry bits — there is no modular wraparound to exploit, and overspending is exactly the statement “the final carry is 1,” which

- $\mathsf{Setup}(1^\lambda, \mathsf{app})$: run MAYO trapdoor generation for the chosen parameter set; expand the whipped forms; let h_{pk} be the domain-separated SHAKE256 hash of the canonical row-major encoding of the expanded whipped forms of P^*; set

$$\mathsf{ctx} \leftarrow \mathsf{SHAKE256}(\text{“context”} \,\|\, \mathsf{frame}(\mathsf{app}) \,\|\, h_{\mathsf{pk}} \,\|\, \text{parameter and profile identifiers} \,\|\, \text{balance width})[0{:}256].$$

  Output $(\mathsf{sk}, \mathsf{pk}, \mathsf{ctx})$.

- Issue, client with (pk, b): sample $\kappa, \rho \leftarrow \{0,1\}^{256}$; compute C from the credential stream; send (b, C, π) where $\pi \leftarrow \Pi.\mathsf{Prove}$ for the relation $\mathsf{C} = \mathsf{C}(\kappa, b, \rho)$ with public $(b, \mathsf{C}, \mathsf{ctx})$. Issuer: verify π (and external authorization); return $\sigma \leftarrow \mathsf{SPre}(\mathsf{sk}, \mathsf{C})$. Client: accept the direct token $T = (\sigma, \kappa, b, \rho)$ iff $P^*(\sigma) = \mathsf{C}$.

- Spend, client with (pk, T, s): sample fresh (κ', ρ'), set $b' = \mathsf{bal}(T) - s$, compute C' and N from T's stream; send $(s, \mathsf{N}, \mathsf{C}', \pi)$ for the spend relation of Section 4.4 (typed for T's kind, settlement tag “spend”). Issuer: if the store already holds a record for N with the same request digest, return the stored response; on a digest conflict, reject. Otherwise verify π, sample $\sigma' \leftarrow \mathsf{SPre}(\mathsf{sk}, \mathsf{C}')$, construct the candidate record $(\mathsf{N}, \text{request digest}, \sigma')$, and *atomically* insert it. If another record won the insertion race, compare digests: replay the winner if its digest matches this request, otherwise reject. Client: accept the direct token $(\sigma', \kappa', b', \rho')$ iff $P^*(\sigma') = \mathsf{C}'$.

- $\mathsf{SpendDeferred}$: as Spend with settlement tag “deferred-return-spend”, except the issuer, after verifying π, first picks $t' \le s$, then samples $\sigma' \leftarrow \mathsf{SPre}(\mathsf{sk}, \mathsf{D}(\mathsf{ctx}, \mathsf{C}', t'))$, constructs the candidate record containing (t', σ'), and atomically inserts it; if another record won the race, it replays the winner — including the winner's original t' — when digests match and rejects otherwise. Client: accept the deferred-return token $(\sigma', \kappa', b', \rho', t')$ iff $t' \le s$ and $P^*(\sigma') = \mathsf{D}(\mathsf{ctx}, \mathsf{C}', t')$.

- $\mathsf{VerifyToken}(\mathsf{pk}, T)$: check that T's stored context equals the context derived from pk; that its credential kind and top-up obey the format's convention (a direct token carries no top-up); that the effective balance does not overflow; recompute T's target (C or D) from its opening; and check $P^*(\sigma) =$ target. (The anonymity game relies on honest clients performing *all* of these checks.)

Figure 4: The VOLE-ACT algorithms. Every request and response carries the credential-kind and settlement-mode tags; request digests bind the full typed request bytes under ctx.

the circuit rejects. The issuer verifies, signs the fresh digest ($\sigma' \leftarrow \mathsf{SPre}(\mathsf{sk}, \mathsf{C}')$, or the nested target for a deferred settlement), and atomically stores the request digest and response under N; exact retransmissions replay the stored response, so a network failure cannot burn a balance.

5 The proof circuit

The relation above is almost entirely SHAKE256: a direct-input spend proves two hidden sponge evaluations (old stream, fresh stream), a deferred-input spend proves three (the nested target costs one more). Everything else — the MAYO equation and the balance arithmetic — is quadratic. This section describes the three techniques that make the circuit small.

5.1 Degree-16 Keccak checkpointing

Keccak-$f[1600]$'s round function is degree 2 (the χ step is the only nonlinearity) [13]. The naive circuit commits all 1600 state bits after every round: 24 rounds $\times$ 1600 witness bits, with degree-2

constraints between layers. But witness bits are the dominant cost in a VOLE-style proof (each consumes a VOLE coordinate, enlarging both the correction data and every leaf-PRG expansion), while raising the *degree* of the batched QuickSilver check is cheap: a maximum degree d costs d published field elements and $d-1$ mask VOLEs *in total*.

We therefore commit the state only every *four* rounds. Between checkpoints the circuit carries symbolic polynomial expressions: after rounds $1, 2, 3, 4$ the expressions have degree $2, 4, 8, 16$, and the checkpoint constraint asserts (per bit) that the degree-16 expression equals the next committed state — a degree-16 polynomial constraint, the largest the instantiation accepts. This quarters the per-permutation witness from 24×1600 to $6 \times 1600 = 9600$ bits. One permutation dominates each hidden SHAKE evaluation, so a direct spend's witness is 21,712 bits and a deferred spend's 31,504 — versus roughly 79,300 and 117,900 under per-round checkpointing. The prover-side cost is the coefficient-vector products of the expression layer, about 1.14 million $\mathbb{F}_{2^{128}}$ multiplications per permutation.

5.2 The quadratic-system constraint for MAYO

Verifying $P^*(\sigma) = y$ means checking $m = 64$ quadratic equations

$$\sum_{r \le c} G_a[r, c]\ \sigma_r \sigma_c \ = \ y_a, \qquad a \in [m],$$

over the *same* $\binom{325}{2} = 52{,}650$ products $\sigma_r \sigma_c$. Materializing each product as a committed wire would multiply the witness; instead the proof system supports a *deferred quadratic system*: the prover records the values and tags of every term factor, and at challenge time the m equations are folded into a single QuickSilver degree-2 check by random weights $\varphi_a \in \mathbb{F}_{2^{128}}$ drawn from the transcript, with coefficients lifted through the embedding ι. An unsatisfied equation survives the fold except with probability $2^{-\lambda}$. The committed signature nibbles enter as bit wires ($4k_w n = 1296$ bits). The same mechanism folds the 64 quadratic carry equations of each balance addition.

5.3 Balance arithmetic in $\mathbb{F}_2$

Additions are proven bitwise with committed carry vectors: $a_i + b_i + c_i = s_i$ linearly (over $\mathbb{F}_2$, addition is XOR — an early design error we caught was writing balance equations additively over the tag field, where they say nothing about integers), and the carry recurrence $c_{i+1} = a_i b_i + c_i(a_i + b_i)$ quadratically via the folded system. Asserting the final carry to be zero makes every balance equation an exact statement about 64-bit unsigned integers, so conservation is enforced bit-exactly rather than modulo anything.

6 Security analysis

We state the intended results as conditional claims against the definitions of Section 3; no complete reduction is claimed, and where the implementation's own analysis states sharper caveats, those govern.

Claim 1 (Fiscal soundness, conditional)**.** *In the ideal-XOF model of Remark 4, under Assumption 1, adaptive multi-theorem knowledge soundness (extractability) of the VOLEitH argument for* both *the issuance and the spend relations, and a linearizable, non-rollback nullifier store, VOLE-ACT satisfies Definition 3.*

Discussion and error terms. Conditionally on extraction, every accepted operation yields an opening and a MAYO preimage for an admissible target, correct nullifier derivation, and exact balance arithmetic; over-redemption then requires either one more admissible preimage than the issuer produced (Assumption 1) or a *target collision*. The collision cases are: two credential-stream openings with equal C; two deferred wrappers with equal D output; and a cross-domain collision $\mathsf{C} = \mathsf{D}(\cdot)$ — the domain tags separate the hash inputs, but all three remain generic collision events in the same $4m$-bit output space and must be counted. A fourth, operational collision event: the retry store retains a 256-bit *request digest* rather than the full request, so exact-retry semantics additionally assume collision resistance of that digest (or full-request comparison), and the redeemed service itself — not merely the token response — must be tied idempotently to the first durable record. Issuance soundness matters because Assumption 1 feeds the oracle only opening-accompanied targets: without it, a malicious issuance request could obtain a preimage for an unopened target. The dominant known terms and generic ceilings include (this is not yet a complete advantage expression — proof-attempt counts, Q_H factors, vector-commitment and consistency terms, and composition loss are unquantified):

- The degree-16 batched QuickSilver check contributes up to $17/2^{128} \approx 2^{-123.9}$ per proof attempt, before vector-commitment, consistency-hash, Fiat–Shamir, and composition terms.
- The credential digest is a collision domain of only $4m = 256$ bits. Two openings $(\kappa, b, \rho) \neq (\kappa', b', \rho')$ with the same prefix C let the *one* issuer-supplied preimage σ be presented under both — their suffix nullifiers will generally differ — double-spending, and with distinct balances creating value. This is generic collision search: about 2^{128} classically, and query complexity $\Theta(2^{256/3}) \approx 2^{85.3}$ for an ideal quantum collision adversary [14] (under BHT's demanding memory model). Deployments requiring a larger quantum collision margin need a wider digest, hence a larger m.

Honest double spends are prevented operationally: a repeated credential reveals the same deterministic nullifier, whose conflicting reuse the store refuses while exact retransmissions are replayed from the stored record — the atomic retry semantics that also make honest retransmission safe.

Claim 2 (Spend anonymity, conditional)**.** *In the ideal-XOF model of Remark 4, assuming the VOLEitH argument is adaptive multi-theorem non-interactive zero knowledge for the chosen-key relation (i.e., against maliciously formed but canonically parsed public keys, in the ROM), VOLE-ACT satisfies Definition 4; the claim does not depend on any MAYO assumption.*

Discussion. A spend reveals $(s, \mathsf{N}, \mathsf{C}', \text{proof})$. The intended argument: the compiled argument is NIZK zero-knowledge in the sense above (the 2λ-bit per-proof salt is multi-instance *hardening*, not by itself a theorem upgrading honest-verifier ZK to this property); C' hides its opening under the fresh 256-bit nonce ρ'; and N is the unrevealed suffix of an XOF stream whose prefix the issuer saw at issuance, pseudorandom given the prefix in the ideal-XOF model. None of these steps is currently backed by a proof for this exact composition, so the claim is conditional on all three. Its MAYO-independence is the design's compartmentalization: a trapdoor recovery or forgery does not help the distinguisher in Definition 4. Outside the game, deployments leak metadata the definition deliberately excludes: spend amounts are public; direct-input and deferred-input proofs differ substantially in size and partition the anonymity set, as do credential kind, settlement mode, request length, timing, network identity, and retry behavior.

Multi-instance hardening. Every PRG and commitment call in the tree layer binds salt and position (after [6]), preventing the known precomputation and batch-amortization attacks on the 128-bit seeds; a concrete multi-user bound still carries adversary-instance-dependent terms. The wide consistency hash removes the 2^{-h} per-chunk cheating strategy. Domain tags separate every hash use; all encodings are canonical and rejected on any alternate serialization.

7 Parameters and evaluation

The construction is implemented as a Rust research prototype (five crates: binary fields, GGM vector commitments, the VOLEitH proof system, MAYO, and the protocol). Table 1 gives wire sizes; tokens are profile-independent and only proof-carrying requests grow with the low-latency profiles. On one Apple M4 Pro, a direct spend proves in 19–20 ms and verifies-and-signs in 6.5–8 ms across profiles (issuance: 8–9 ms and 3–4 ms; deferred-input spends roughly 1.4× a direct spend). Figures are single-operation latencies on an idle machine. The compact profile's proving penalty over low-latency is ≈6–7% while its proofs are 3.8× smaller. Implementation techniques, optimization methodology, and full per-profile measurements are documented in the repository, `https://github.com/SamuelSchlesinger/vole-act` (`docs/IMPLEMENTATION.md`, `docs/BENCHMARKS.md`; the measurements in this paper correspond to commit `65248c8`).

Table 1: Wire sizes (bytes), MAYO_2, $\lambda = 128$.

Object	compact (τ, h)=(16, 8)	balanced (32, 4)	low-latency (64, 2)
Token (either format)	283	283	283
Issue request	29 750	55 286	106 358
Spend request (direct input)	52 990	101 726	199 198
Spend request (deferred input)	72 574	140 894	277 534
Issue / spend response	171 / 179	171 / 179	171 / 179
Public key (expanded map)	106 319	106 319	106 319

8 Related work

VOLE-based designated-verifier ZK begins with Wolverine [8] and Mac'n'Cheese [9]; Quick-Silver [7] gives the degree-d batched check we generalize. VOLE-in-the-Head [4] removes the designated verifier and underlies the FAEST signature candidate [5], whose engineering (GGM trees, salting, consistency checking) we follow, with the hardening of [6]. MAYO [1, 3] descends from unbalanced oil-and-vinegar [12]; [2] is the cautionary tale for the assumption class. The closest post-quantum relative is PoMFRIT [19], a *blind-signature* scheme that likewise combines VOLE-in-the-Head proofs of SHA-3-family circuits with the MAYO trapdoor and analyzes the one-more-preimage property such issuers require — an instructive contrast: PoMFRIT pursues the blindness route with the same toolbox, while VOLE-ACT's commit-then-sign design avoids blindness and adds stateful balances, typed settlement, and deferred returns. Anonymous tokens and credentials span blind signatures [15], CL signatures [16], keyed-verification credentials from algebraic MACs [17], and deployed unit-token systems [18]; VOLE-ACT is a post-quantum stateful-balance member of this family, instantiating the Katz–Schlesinger design [20].

9 Open problems

(1) A complete security reduction: formalizing Definitions 3–4 fully (concurrency, chains, adaptive corruptions) and proving Claims 1 and 2 in the QROM, including a precise formulation of Assumption 1 and its relation to MAYO's standard security. (2) A structured, non-materialized QuickSilver check for the whipped MAYO equation, removing the $O(m(k_w n)^2)$ expanded-form table and enabling the larger MAYO parameter sets (whose wider digests also raise the quantum collision ceiling). (3) Hashing the compact public-key encoding (rather than the expanded forms) into the context — a transcript-breaking cleanup best done before any format freeze. (4) Aggregating or compressing spend proofs; at 53 KiB the compact profile is deployable server-side but remains the dominant bandwidth cost. (5) Hidden spend amounts, which would require in-circuit range checks and re-open the balance-width trade-off.

References

[1] W. Beullens. *MAYO: Practical Post-Quantum Signatures from Oil-and-Vinegar Maps*. SAC 2021.

[2] W. Beullens. *Breaking Rainbow Takes a Weekend on a Laptop*. CRYPTO 2022.

[3] W. Beullens, F. Campos, S. Celi, B. Hess, M. J. Kannwischer. *MAYO: Algorithm Specification (Round-2 document, February 5, 2025)*. NIST PQC Additional Digital Signatures. This work pins the Round-2 specification; MAYO advanced to Round 3 in May 2026. https://pqmayo.org/resources/.

[4] C. Baum, L. Braun, C. Delpech de Saint Guilhem, M. Klooß, E. Orsini, L. Roy, P. Scholl. *Publicly Verifiable Zero-Knowledge and Post-Quantum Signatures from VOLE-in-the-Head*. CRYPTO 2023.

[5] C. Baum, W. Beullens, L. Braun, C. Delpech de Saint Guilhem, M. Klooß, C. Majenz, S. Mukherjee, E. Orsini, S. Ramacher, C. Rechberger, L. Roy, P. Scholl. *FAEST: Algorithm Specification, Version 2.0 (Round-2 document)*. NIST PQC Additional Digital Signatures. This work follows the Round-2 (v2.0) design; FAEST advanced to Round 3 in May 2026. https://faest.info.

[6] C. Baum, W. Beullens, S. Mukherjee, E. Orsini, S. Ramacher, C. Rechberger, L. Roy, P. Scholl. *One Tree to Rule Them All: Optimizing GGM Trees and OWFs for Post-Quantum Signatures*. ASIACRYPT 2024.

[7] K. Yang, P. Sarkar, C. Weng, X. Wang. *QuickSilver: Efficient and Affordable Zero-Knowledge Proofs for Circuits and Polynomials over Any Field*. ACM CCS 2021.

[8] C. Weng, K. Yang, J. Katz, X. Wang. *Wolverine: Fast, Scalable, and Communication-Efficient Zero-Knowledge Proofs for Boolean and Arithmetic Circuits*. IEEE S&P 2021.

[9] C. Baum, A. J. Malozemoff, M. B. Rosen, P. Scholl. *Mac'n'Cheese: Zero-Knowledge Proofs for Boolean and Arithmetic Circuits with Nested Disjunctions*. CRYPTO 2021.

[10] L. Roy. *SoftSpokenOT: Quieter OT Extension from Small-Field Silent VOLE in the Minicrypt Model*. CRYPTO 2022.

[11] O. Goldreich, S. Goldwasser, S. Micali. *How to Construct Random Functions*. Journal of the ACM 33(4), 1986.

[12] A. Kipnis, J. Patarin, L. Goubin. *Unbalanced Oil and Vinegar Signature Schemes*. EUROCRYPT 1999.

[13] NIST. *SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions*. FIPS PUB 202, 2015.

[14] G. Brassard, P. Høyer, A. Tapp. *Quantum Cryptanalysis of Hash and Claw-Free Functions*. LATIN 1998. arXiv:quant-ph/9705002.

[15] D. Chaum. *Blind Signatures for Untraceable Payments*. CRYPTO 1982.

[16] J. Camenisch, A. Lysyanskaya. *An Efficient System for Non-transferable Anonymous Credentials with Optional Anonymity Revocation*. EUROCRYPT 2001.

[17] M. Chase, S. Meiklejohn, G. Zaverucha. *Algebraic MACs and Keyed-Verification Anonymous Credentials*. ACM CCS 2014.

[18] A. Davidson, I. Goldberg, N. Sullivan, G. Tankersley, F. Valsorda. *Privacy Pass: Bypassing Internet Challenges Anonymously*. Proceedings on Privacy Enhancing Technologies 2018(3), 164–180.

[19] C. Baum, M. Beckmann, W. Beullens, S. Mukherjee, C. Rechberger. *Concretely Efficient Blind Signatures Based on VOLE-in-the-Head Proofs and the MAYO Trapdoor (PoMFRIT)*. USENIX Security 2026. IACR ePrint 2026/109.

[20] J. Katz, S. Schlesinger. *Anonymous Credit Tokens*. Design document and Rust implementation, https://github.com/SamuelSchlesinger/anonymous-credit-tokens.